CSA4704 - Deep Learning · Assignment 1

Mathematical and Machine Learning Foundations of the AgriVision Crop-Disease Screening System

Name: Harini. R

Register no: 192324108

Dataset and Setup

- Dataset: PlantVillage (grayscale), all 10 tomato classes, 18,160 images. I used 10 classes instead of 3 because the 3-class subset has only 4,500 images, which is not enough for the 5,000 and 10,000 scales.
- Preprocessing: resized 256×256 → 64×64, pixel values divided by 255, flattened to 4,096 features, stratified 70/15/15 split (12,712 train / 2,724 validation / 2,724 test).
- Scales used: 1,000 → 5,000 → 10,000 → 18,160.
- All answers are rounded to 4 decimal places. Information quantities are in bits ($\log_2$). Random seed = 47.
- Code and outputs are given in the Appendix.

Class counts: Bacterial spot 2127, Early blight 1000, Late blight 1909, Leaf mold 952, Septoria 1771, Spider mites 1676, Target spot 1404, Yellow Leaf Curl Virus (YLCV) 5357, Mosaic 373, Healthy 1591.

Task 1 – Problem Structure and Data Representation

1.1 Learning task components

Component	AgriVision	Effect on the solution
Task	Classify a leaf image into 1 of 10 classes	Needs softmax output and cross-entropy loss
Input	64×64 grayscale image = 4,096 pixel values in [0,1]	High dimension, only ~3 training images per feature
Output	Class label 0–9 (or one-hot vector)	Classification, not regression
Performance measure	Accuracy, precision, recall, F1, confusion matrix	Classes are imbalanced (YLCV 29.5%, Mosaic 2.1%), so accuracy alone is misleading

1.2 Data shapes

Object	Shape	Why it matters
One pixel / label	scalar	Single value

Object	Shape	Why it matters
One image	vector (4096,) or matrix (64,64)	Flattening loses spatial neighbours: pixel 2048 (row 32, col 0) and pixel 2112 (row 33, col 0) are neighbours in the image but 64 positions apart in the vector
Mini-batch of 32	matrix (32, 4096) or tensor (32, 1, 64, 64)	Forward pass is one matrix multiply: (32×4096)(4096×10) = (32×10)
Full dataset	matrix (18160, 4096)	283.75 MB in float32
Labels	vector (18160,) or one-hot (18160, 10)	One-hot rows are probability distributions, so cross-entropy can be used

1.3 Norms

Formulas: $\|x\|_1 = \Sigma|x_i|$, $\|x\|_2 = \sqrt{(\Sigma x_i^2)}$, $\|x\|_\infty = \max|x_i|$

Hand example, x = [0.2, 0.5, 0.9, 0.4]:

- L1 = 0.2 + 0.5 + 0.9 + 0.4 = 2.0000
- L2 = $\sqrt{(0.04 + 0.25 + 0.81 + 0.16)} = \sqrt{1.26} = 1.1225$
- $L\infty = 0.9000$

On my data:

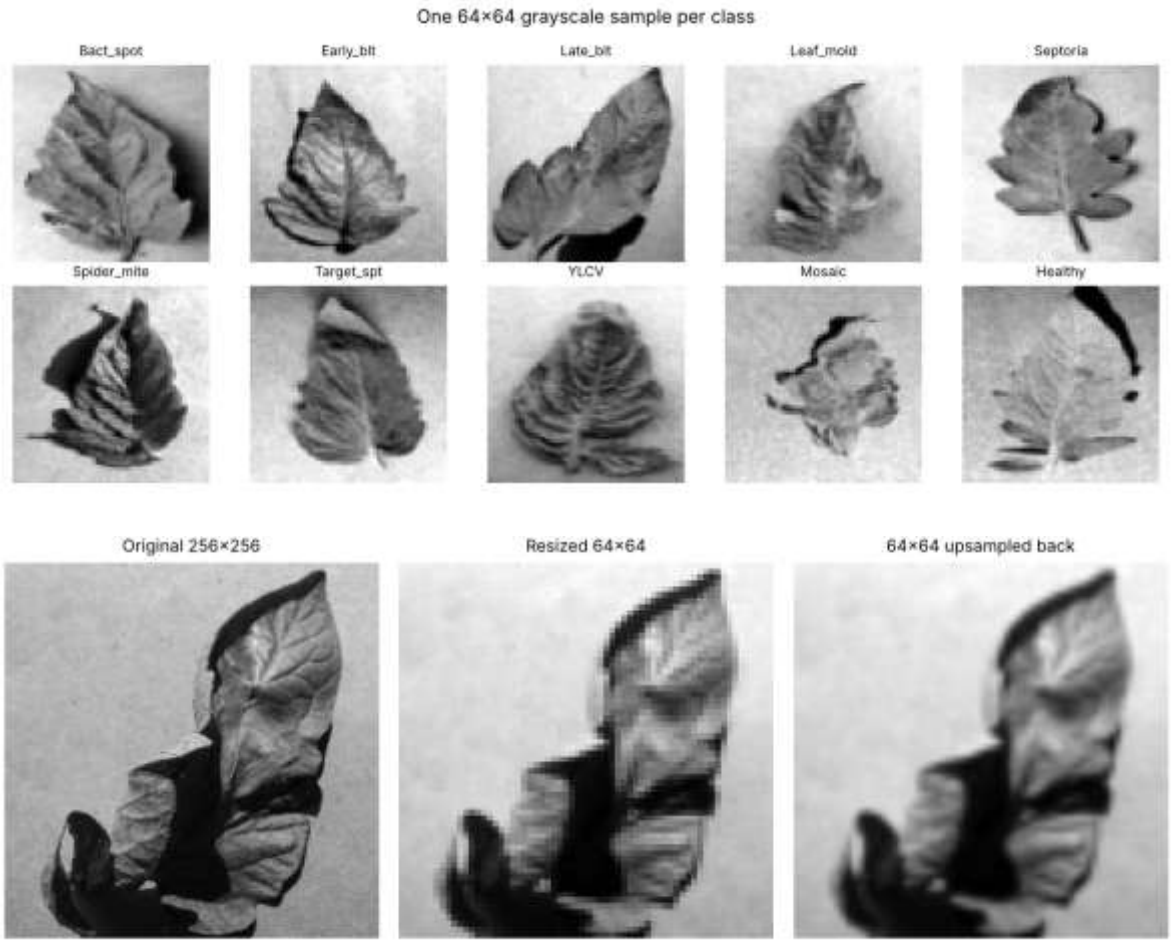
Change to image	L1	L2	L∞
One pixel changed by 0.5 (small lesion)	0.5000	0.5000	0.5000
All pixels brighter by 0.02 (lighting)	81.9200	1.2800	0.0200

- L1 counts every changed pixel, which makes it useful for sparse (L1) regularisation.
- L2 is used in Euclidean distance (k-NN) and PCA. Here a lighting change (1.28) scores higher than a real lesion (0.50), which hurts k-NN.
- L∞ only sees the biggest single change, which makes it useful for robustness checks.

1.4 Effect of preprocessing

- Resizing keeps only 6.25% of pixels (resize round-trip error RMSE = 0.0495), so small lesions get blurred out.
- Grayscale removes colour cues (yellowing, mosaic patterns).
- Normalisation (÷255) loses no information but keeps values numerically safe.
- Flattening keeps all values but loses spatial structure.
- The stratified split keeps class proportions equal (YLCV: 0.2950 / 0.2952 / 0.2948). Only 0.11% of test images have an exact duplicate in train.

Meaning for AgriVision: the biggest information losses are colour, fine detail and spatial structure, which are exactly what disease spots depend on.



Task 2 – Feature Space using Linear Algebra

Formulas:

- Covariance: $\Sigma = X_c^T X_c / (n-1)$
- Eigendecomposition: $\Sigma = V \Lambda V^T$
- SVD: $X_c = U S V^T$, with $\lambda_i = s_i^2 / (n-1)$
- Explained variance: $EV(k) = \Sigma_{i=1}^{k} \lambda_i / \Sigma \lambda_i$

2.1 Hand-worked 2×2 example (covariance of two neighbouring centre pixels)

$\Sigma = [[0.018, 0.016], [0.016, 0.018]]$

Step 1: $\det(\Sigma - \lambda I) = (0.018 - \lambda)^2 - 0.016^2 = 0$
Step 2: $\lambda_1 = 0.018 + 0.016 = 0.0340$, $\lambda_2 = 0.018 - 0.016 = 0.0020$
Step 3: For λ_1, $-0.016 v_1 + 0.016 v_2 = 0$, so $v_1 = v_2 \rightarrow$ eigenvector [0.7071, 0.7071]. For $\lambda_2 \rightarrow$ [0.7071, −0.7071].
Step 4: Explained variance of the first direction = 0.034 / 0.036 = 0.9444
Step 5: Singular values (n = 12,712): $s_1 = \sqrt{(12711 \times 0.034)} = 20.7888$, $s_2 = \sqrt{(12711 \times 0.002)} = 5.0420$

Meaning: neighbouring pixels are highly correlated (r = 0.9012), so pixels are redundant. This redundancy is what PCA uses.

2.2 Full spectrum (4096×4096)

- Top eigenvalues: 17.1232, 9.4852, 8.6564, 8.0191, 2.2863, …
- Variance explained: PC1 = 0.2072, top 10 = 0.6342, top 50 = 0.7994, top 100 = 0.8594
- Components needed: 80% → 51, 90% → 169, 95% → 384, 99% → 1277
- SVD check (n = 1000): $s^2/(n-1)$ = 17.7969, 9.7455, … matches the eigenvalues exactly.

The spectrum drops fast and then has a long flat tail.

2.3 Reconstruction error

k	10	20	50	100	200	1000
Relative error (validation)	0.3717	0.2958	0.2064	0.1464	0.0947	0.0208

The trade-off point is around k = 50 (≈80% variance, 82× compression); beyond it, gains are small. I use k = 50 for the experiments.

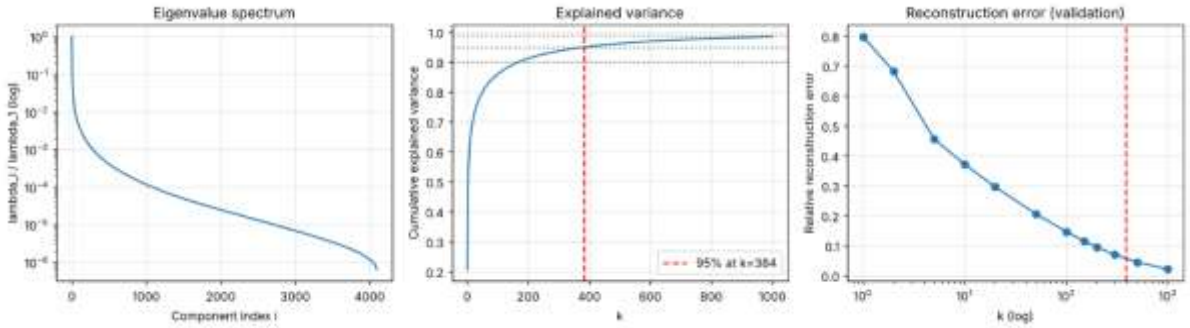
2.4 Leading components

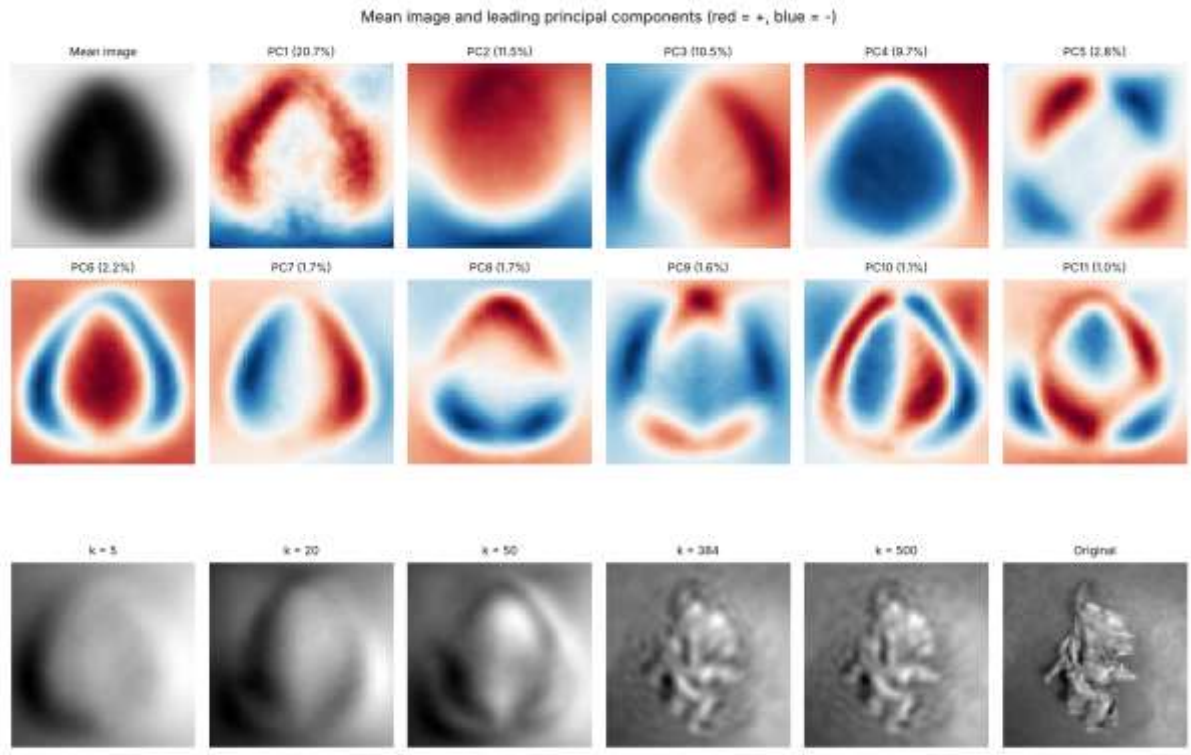
- PC1 = overall brightness (correlation with mean brightness = 0.9984)
- PC2 = top vs bottom position
- PC3 = left vs right tilt
- PC4 = leaf size/shape

The largest components capture lighting and position, not disease.

2.5 Cost

SVD cost is O(n·d·min(n,d)). Measured times: n = 1000 → 0.7630 s, 5000 → 18.2073 s, 12712 → 20.7421 s. Growing n from 1000 to 5000 was predicted to take 20.48× longer; observed 23.86×.





Task 3 – Uncertainty using Information Theory

Formulas:

- Entropy: $H(p) = -\Sigma p_i \log_2 p_i$
- Cross-entropy: $H(p,q) = -\Sigma p_i \log_2 q_i$
- KL divergence: $KL(p\|q) = \Sigma p_i \log_2 (p_i/q_i)$
- Identity: $H(p,q) = H(p) + KL(p\|q)$

3.1 Entropy and imbalance

Hand example (Early 1000, Late 1909, Healthy 1591):
p = [0.2222, 0.4242, 0.3536]
H = 0.4822 + 0.5248 + 0.5303 = 1.5373 bits (maximum $\log_2 3 = 1.5850$)

10 classes: H(p) = 3.0300 bits vs maximum $\log_2 10 = 3.3219$ bits. Imbalance removes 0.2919 bits of uncertainty. Always predicting YLCV already gives 29.50% accuracy, so that is the baseline any model must beat.

3.2 Cross-entropy and KL across models

Model	Accuracy	Cross-entropy (bits)	% test images with q(true) = 0
LogReg (raw)	0.3979	4.1657	0.00
k-NN (raw)	0.4626	3.4791	27.75
Random Forest (raw)	0.6630	1.7303	0.07
LogReg (PCA-50)	0.4916	2.3121	0.00

Model	Accuracy	Cross-entropy (bits)	% test images with q(true) = 0
k-NN (PCA-50)	0.6171	2.2472	15.46
Random Forest (PCA-50)	0.6200	1.8675	0.00

- Raw LogReg (4.1657) is worse than just predicting the class prior (3.0300), because it is over-confident when wrong.
- k-NN gives zero probability to the true class 27.75% of the time, so its probabilities are not reliable.
- Random Forest has the lowest cross-entropy, so its probabilities are the best calibrated.

3.3 Why KL is asymmetric

p = [0.2222, 0.4242, 0.3536], q = [0.05, 0.60, 0.35] (a model that almost ignores Early blight)

KL(p‖q) = 0.4781 − 0.2122 + 0.0052 = 0.2712 bits
KL(q‖p) = −0.1076 + 0.3001 − 0.0052 = 0.1874 bits

- KL(p‖q) is weighted by the true distribution, so it punishes missing a real class heavily (Early blight term = 0.4781).
- KL(q‖p) is weighted by the model, so it punishes over-predicting a class (Late blight term = 0.3001).

3.4 Why minimising cross-entropy = minimising KL

H(p,q) = H(p) + KL(p‖q). H(p) does not depend on the model, so both have the same gradient and the same minimum.
Check: H(p,q̄) = 3.0323 = 3.0300 + 0.0023 ✓

Meaning: cross-entropy shows confident wrong answers that accuracy hides, and it favours the Random Forest.

Task 4 – Numerical Computation Issues

4.1 Naive softmax failure

softmax(z)$_i$ = e^z$_i$ / Σe^z$_j$

- For z = [1000, 999, 995]: e^{1000} overflows (float64 limit ≈ e^709.78), giving [inf, inf, inf] → NaN.
- For z = [−1000, −999, −995]: everything underflows to 0, giving 0/0 → NaN.
- In float32 the limit is about 88.72. With raw 0–255 pixels, my logits ranged from −18798 to 20612, and all 200 test rows gave NaN.

4.2 Fix

Step 1: c = max(z) = 1000, so z − c = [0, −1, −5]
Step 2: e^(z−c) = [1.0000, 0.3679, 0.0067], sum = 1.3746
Step 3: softmax = [0.7275, 0.2676, 0.0049]

Why the answer doesn't change: $e^{(z_i-c)} / \Sigma e^{(z_j-c)} = e^{z_i} / \Sigma e^{z_j}$, because $e^{(-c)}$ cancels. Why it no longer fails: the largest exponent is now $e^0 = 1$, and the sum is at least 1.

Log-sum-exp: LSE = 1000 + ln(1.3746) = 1000.3182, so log-softmax = [−0.3182, −1.3182, −5.3182].

4.3 Condition number

$\kappa = \lambda max / \lambda min$

Features	κ
Raw pixels	1.553×10^6
Standardised pixels	1.300×10^6
PCA-50	113.4372
PCA whitened	1.0000

A large κ means the learning rate is limited by the steepest direction, so the flat directions barely move and training is slow. Raw LogReg took 57.58 s and reached only 0.3979 accuracy; PCA-50 LogReg took 3.56 s and reached 0.4916.

Task 5 – Baseline Learning Strategies

	Logistic regression	k-NN (k=5)	Random Forest (100 trees)
Type	Linear parametric	Non-parametric	Tree ensemble
Hypothesis space	Linear boundaries	Any shape (nearest neighbours vote)	Axis-aligned splits, averaged
Inductive bias	Classes are linearly separable	Nearby images have the same label	Thresholds on individual features matter

Test results (full data):

Features	LogReg	k-NN	Random Forest
Raw (4096)	0.3979	0.4626	**0.6630**
PCA-50	0.4916	0.6171	0.6200
PCA-95% (380)	0.4813	0.5007	0.5033

Why they differ:

- **LogReg:** a lesion can appear anywhere, which a linear model cannot capture. On raw pixels it also overfits (40,970 weights for 12,712 images).
- **k-NN:** raw distances are dominated by lighting; the nearest neighbour has the same label only 47.91% of the time.
- **Random Forest:** it selects useful pixels by itself, so it does best on raw pixels.

Effect of compression:

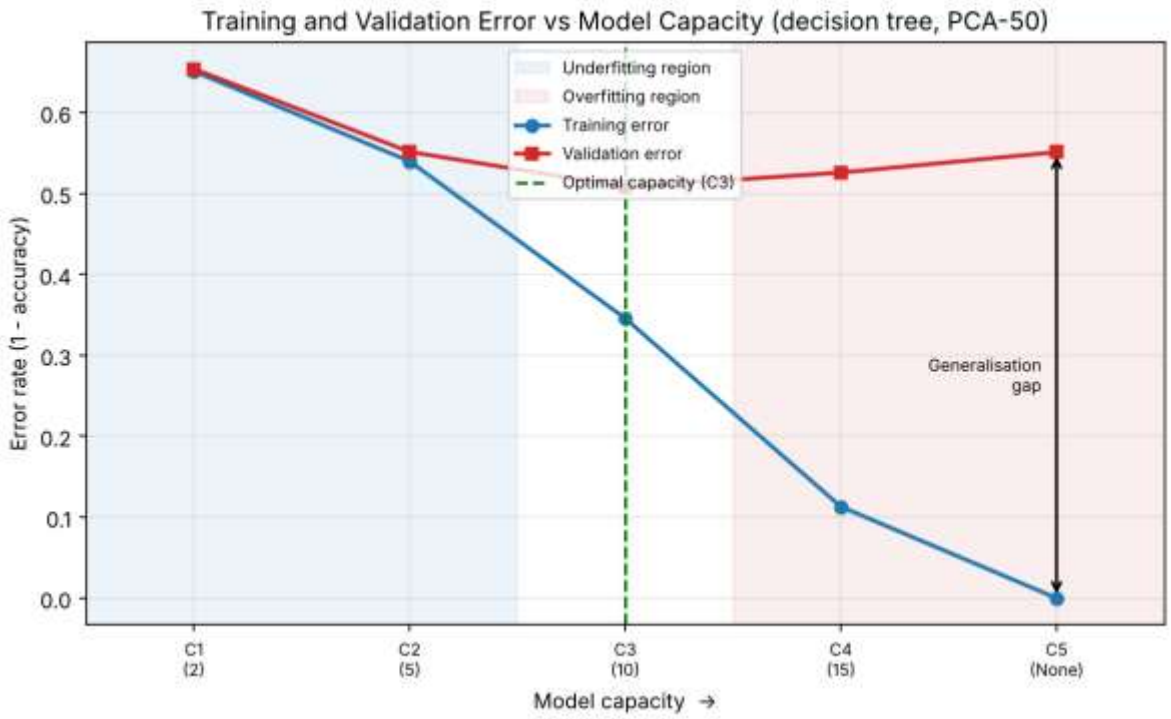
- k-NN gains the most (+0.1545), because PCA removes noisy dimensions from the distance.
- LogReg gains (+0.0937): fewer parameters and better conditioning.
- Random Forest loses (−0.0430), because PCA mixes local pixels into global features.
- PCA-95% is worse than PCA-50, because the extra components add mostly noise.

Task 6 – Capacity, Overfitting and Underfitting

Formula: generalisation gap = validation error − training error

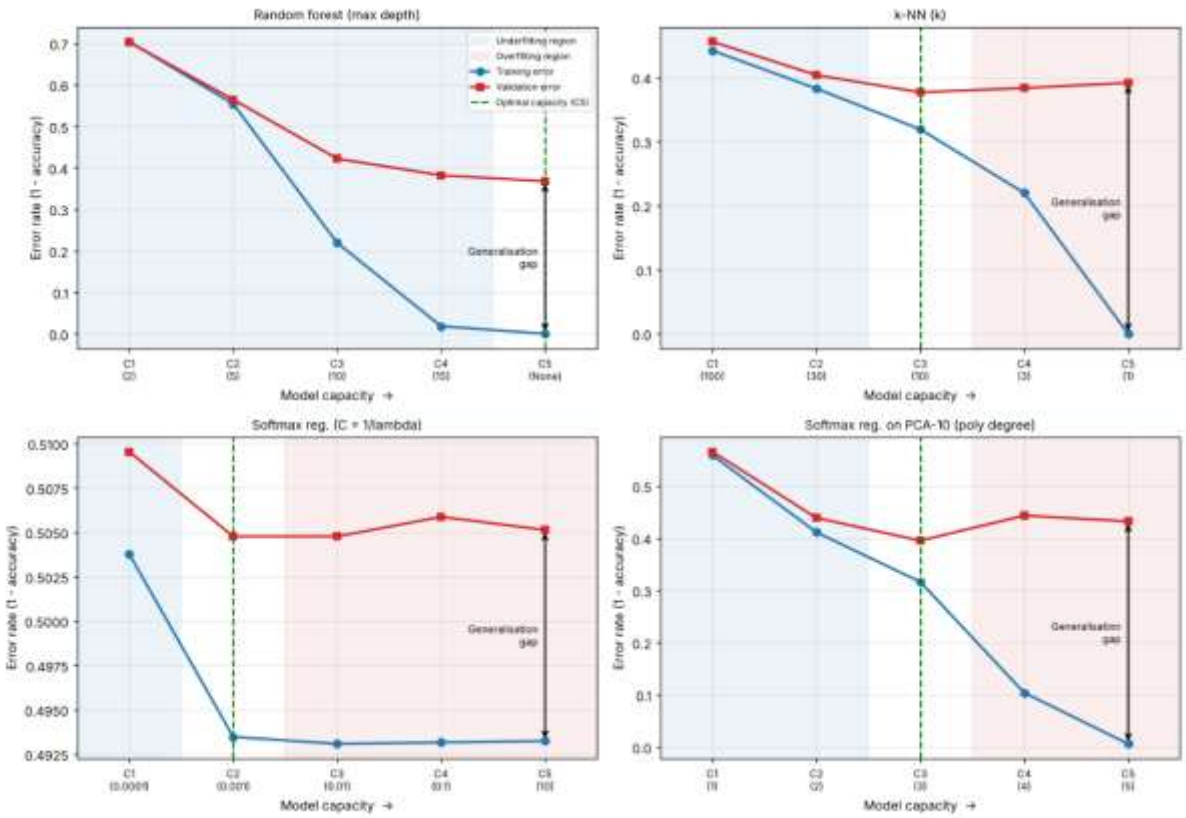
Decision tree on PCA-50 (capacity = max depth):

	C1 (2)	**C2 (5)**	**C3 (10)**	**C4 (15)**	**C5 (None)**
Train error	0.6509	0.5397	0.3456	0.1130	0.0000
Val error	0.6538	0.5514	**0.5081**	0.5257	0.5510
Gap	0.0029	0.0117	0.1625	0.4127	0.5510



Other capacity controls:

- k-NN (k = 100 → 1): best val error 0.3770 at k = 10; gap grows 0.0138 → 0.3917.
- Polynomial degree 1 → 5 (LogReg on PCA-10): best val error 0.3954 at degree 3; gap grows 0.0066 → 0.4263.
- Regularisation strength: val error almost flat (0.5048–0.5095), because the model is already limited by bias.



Regions:

- **Underfitting (C1–C2):** train and validation errors are both high and close. The model is too simple; a depth-2 tree has only 4 leaves for 10 classes. This is high bias.
- **Optimal (C3):** lowest validation error.
- **Overfitting (C4–C5):** training error goes to 0 but validation error rises, because the model memorises noise in individual images. This is high variance.
- The gap increases steadily with capacity.

Test-set selection bias (k-NN, 20 repeats):

Method	Error
k chosen on test set (reported)	0.4917
True error of that choice	0.5314
k chosen on validation set	0.5297
k chosen by 5-fold CV	0.5160

Choosing k on the test set picks the lucky value, so the reported error is 0.0397 too optimistic. A validation set or cross-validation gives an honest estimate.

Task 7 – Bias–Variance and Estimation

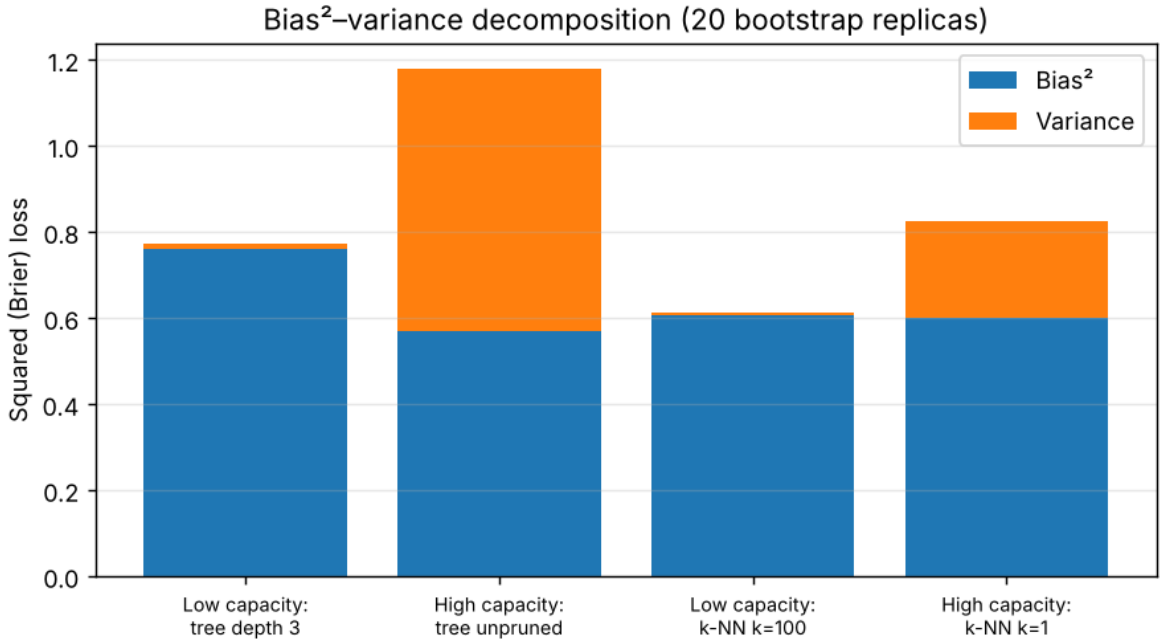
Formulas:

- $\text{Bias}^2 = \|\bar{P} - y\|^2$

- Variance = average $\|P_\beta - \bar{P}\|^2$ over bootstrap models
- Error = Bias² + Variance + Noise

Bootstrap results (20 replicas, PCA-50):

Model	Bias²	Variance	Bias² + Var	Measured error
Tree depth 3 (low capacity)	0.7602	0.0114	0.7716	0.7716
Tree unpruned (high capacity)	0.5696	0.6083	1.1779	1.1779
k-NN k=100 (low capacity)	0.6064	0.0064	0.6128	0.6128
k-NN k=1 (high capacity)	0.6002	0.2254	0.8255	0.8255



- Low-capacity models are dominated by bias; high-capacity models by variance.
- For squared error, bias² + variance matches the measured error exactly, because noise cannot be separated from bias. I estimated noise from near-duplicate image pairs, and their labels disagreed 0.0000 of the time, so label noise is close to zero.
- For 0-1 loss there is a gap. For the unpruned tree, bias + variance = 0.8973 but the measured error is 0.5890, because variance on already-wrong predictions can sometimes make them right.

MLE vs MAP for θ = P(Early blight), prior Beta(2, 18):

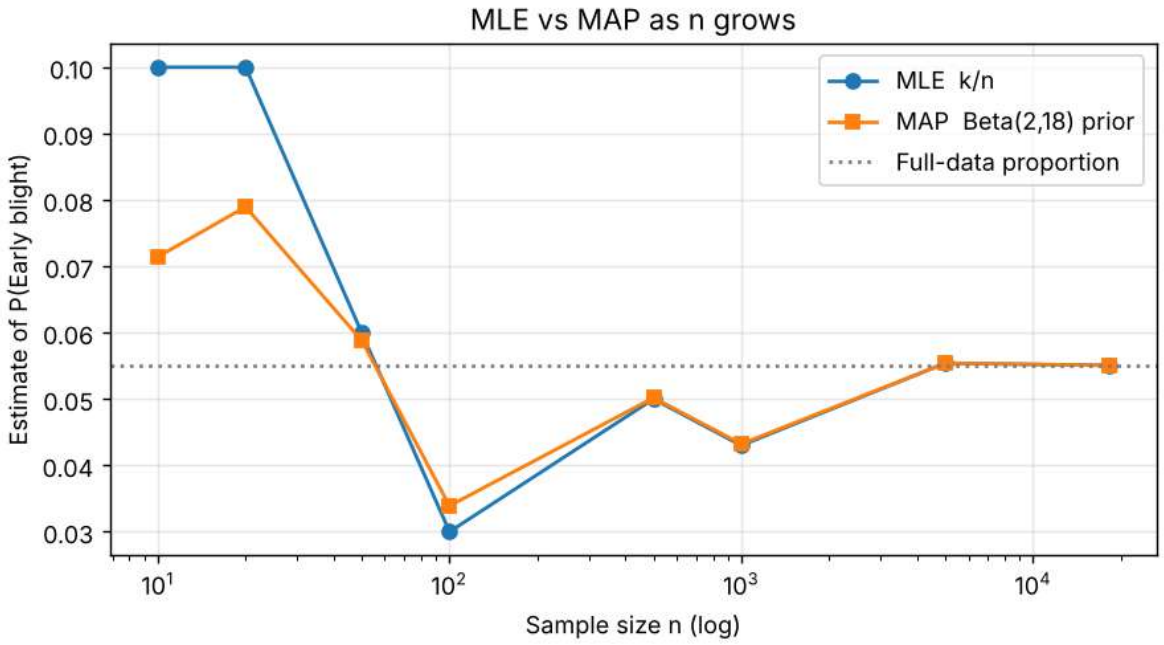
Formulas: MLE = k/n, MAP = $(k + \alpha - 1)/(n + \alpha + \beta - 2)$

Hand example (n = 20, k = 2):

- MLE = 2/20 = 0.1000
- MAP = 3/38 = 0.0789
- Prior weight = 18/38 = 0.4737

n	MLE	MAP	Prior weight
10	0.1000	0.0714	0.6429
100	0.0300	0.0339	0.1525
1000	0.0430	0.0432	0.0177
18160	0.0551	0.0551	0.0010

The prior matters a lot for small n and almost nothing for large n. At n = 10, the MLE gives 0 in 55.76% of samples; the MAP never does.



Task 8 – Gradient-Based Optimisation (Softmax Regression from Scratch)

Model: $z = W^T x + b$, $p_k = e^{z_k} / \Sigma e^{z_j}$, loss $L = -\Sigma y_k \ln p_k$

Gradient derivation:
Step 1: $\ln p_k = z_k - \ln \Sigma e^{z_j}$
Step 2: $L = -\Sigma y_k z_k + \ln \Sigma e^{z_j}$ (since $\Sigma y_k = 1$)
Step 3: $\partial L/\partial z_k = -y_k + p_k = p_k - y_k$
Step 4: $\partial L/\partial W = x(p - y)^T$, $\partial L/\partial b = p - y$
Step 5: For a batch: $\nabla W = (1/m)X^T(P - Y)$

The update is driven by the error $(p - y)$ times the input x. Correct confident predictions give almost no update. The gradient check matched to 8 decimals, and the initial loss was 2.3009 $\approx \ln 10$.

Batch vs SGD vs mini-batch (30 epochs):

Method	Updates	Time/epoch	Final loss	Val acc
Batch GD (η=0.1)	30	0.0125 s	1.9198	0.4659

Method	Updates	Time/epoch	Final loss	Val acc
SGD (η=0.001)	381,360	0.63 s	1.5401	0.4982
Mini-batch 32 (η=0.1)	11,940	0.0357 s	1.5543	0.4978

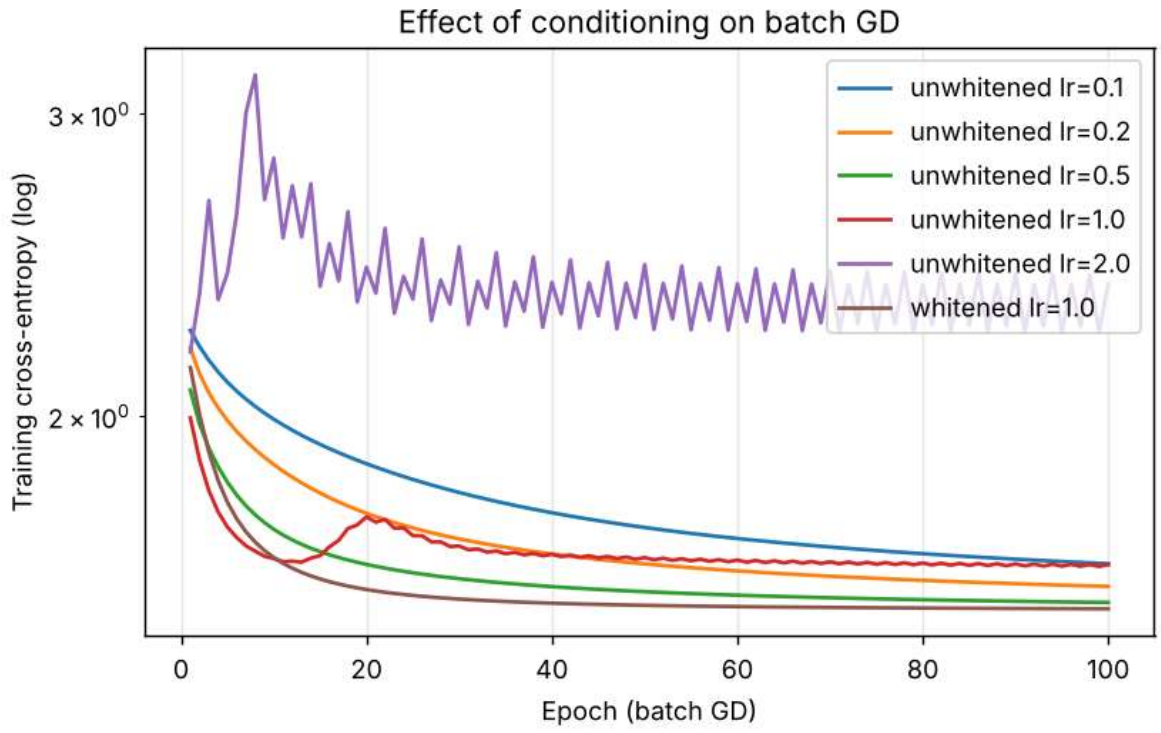
- Batch GD is smooth but slow (only 1 update per epoch).
- SGD is very noisy; with η = 0.1 it never settled (loss 3.4540).
- Mini-batch converged fastest at low cost.

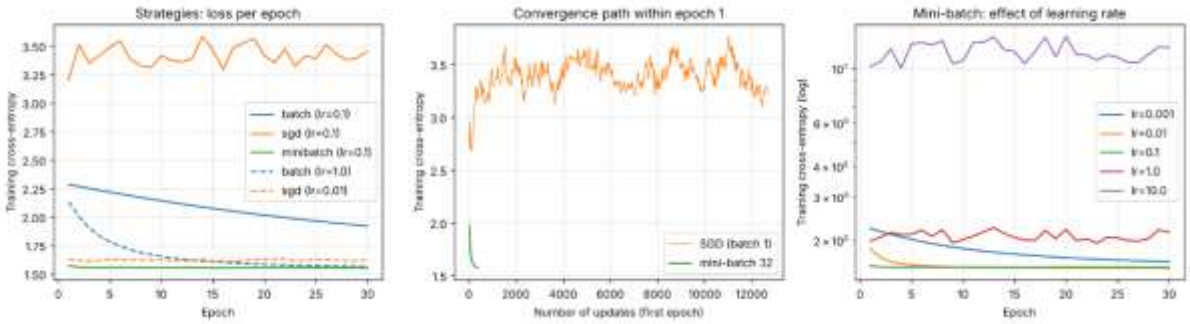
Learning rates (mini-batch):

η	Final loss	Behaviour
0.001	1.6369	Slow but stable
0.01	1.5427	Best
0.1	1.5543	Fast
1.0	2.1573	Oscillates
10.0	12.1989	Diverges

Each step multiplies the error by $(1 - \eta\lambda)$. If $|1 - \eta\lambda| > 1$, training diverges.

Link to conditioning: on unwhitened PCA-50 (κ = 113.4372), training became unstable between η = 0.5 and 1.0. On whitened features (κ = 1), η = 1.0 was stable and reached the lowest loss (1.5436).





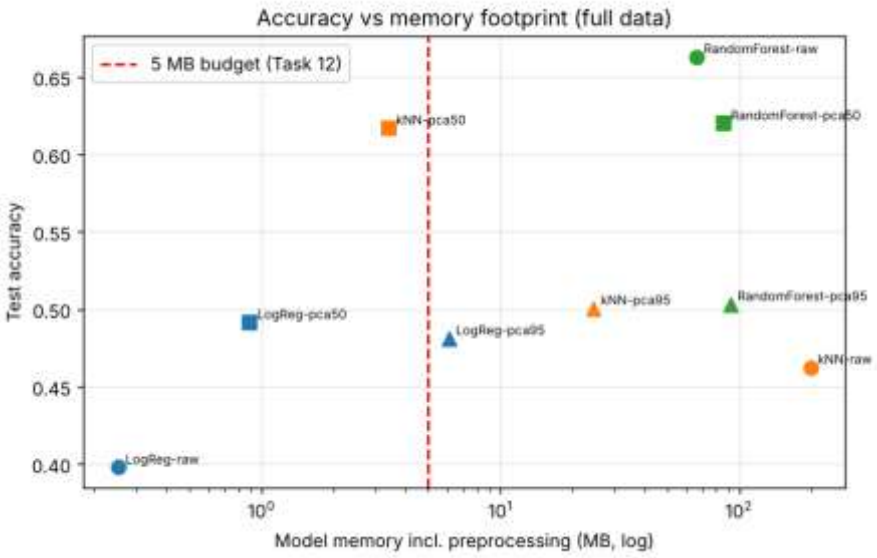
Task 9 – Experimental Performance

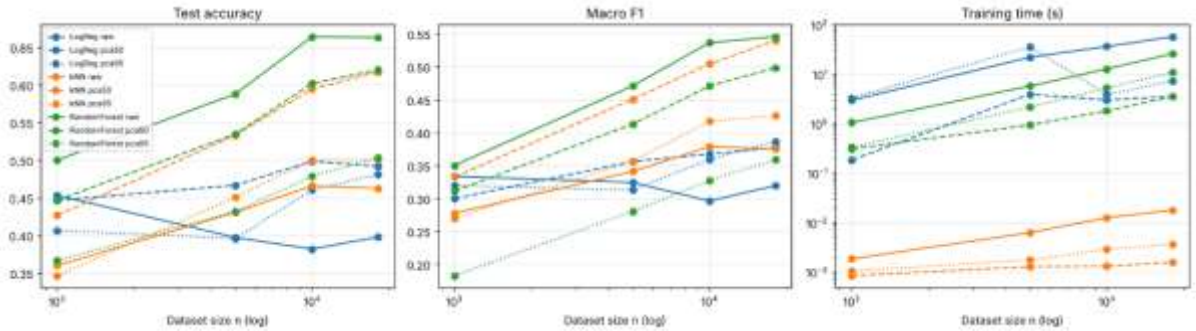
Results across scales (accuracy / macro-F1):

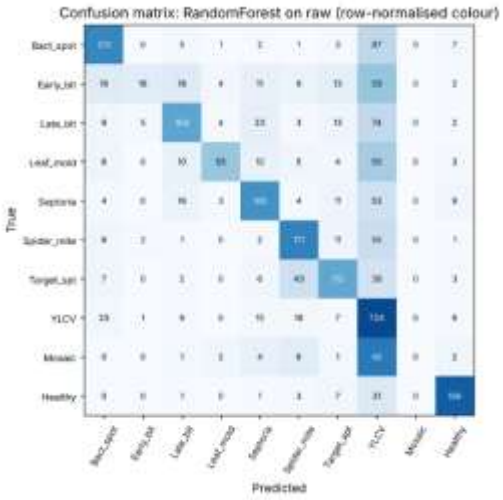
n	LogReg PCA-50	k-NN PCA-50	RF raw
1,000	0.4467 / 0.2997	0.4267 / 0.3325	0.5000 / 0.3503
5,000	0.4667 / 0.3561	0.5333 / 0.4508	0.5880 / 0.4712
10,000	0.4987 / 0.3677	0.5947 / 0.5046	0.6640 / 0.5368
18,160	0.4916 / 0.3771	0.6171 / 0.5401	0.6630 / 0.5459

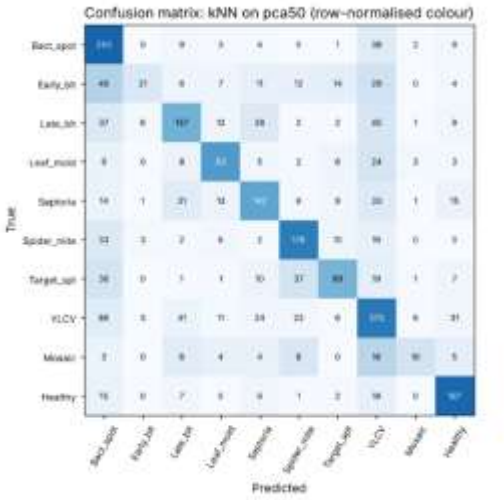
Cost (full data):

Model	Train time	Inference	Parameters	Memory
LogReg raw	57.578 s	0.0041 ms	40,970	0.251 MB
LogReg PCA-50	3.561 s	0.0003 ms	510	0.895 MB
k-NN raw	0.018 s	1.4487 ms	52.1 M stored	198.817 MB
k-NN PCA-50	0.002 s	0.0601 ms	635,600	3.415 MB
RF raw	26.164 s	0.0349 ms	481,904 nodes	66.312 MB









Confused classes (RF):

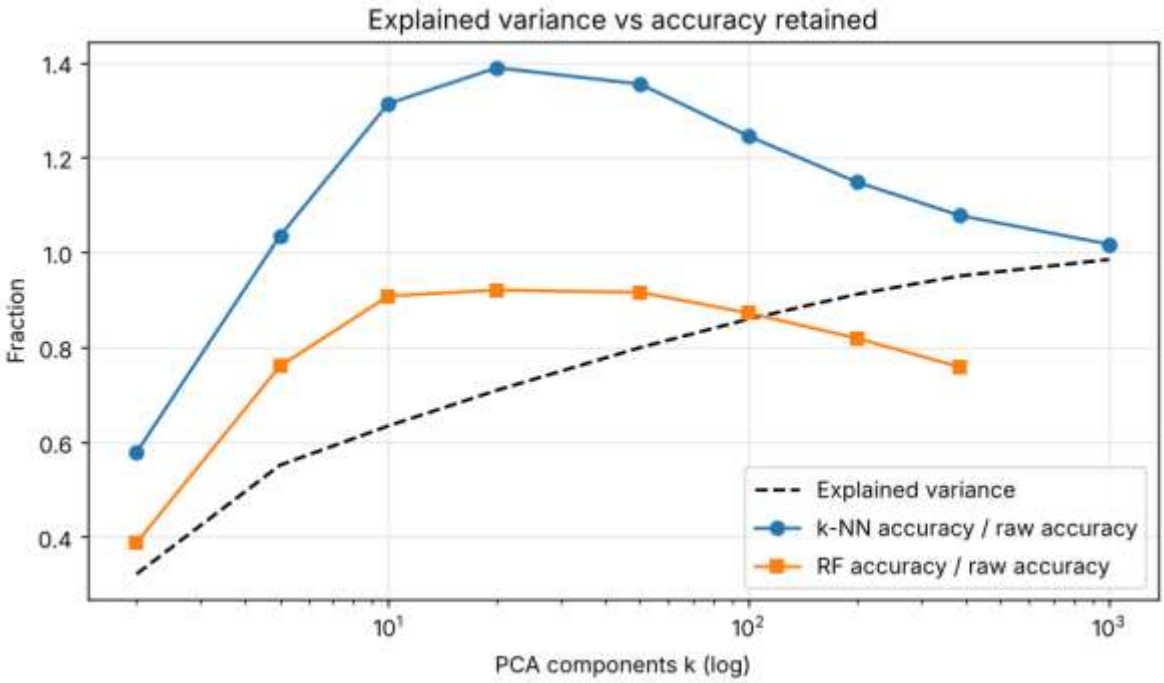
- Most errors are predicted as YLCV, the largest class (e.g. Early blight → YLCV 39.3%).
- Early blight recall is only 0.1067 and Mosaic 0.0000.
- Reason: Early blight, Bacterial spot and Septoria all look like dark spots, and their distinguishing texture and colour are lost at 64×64 grayscale. Mosaic is the rarest class and depends on colour.

Patterns:

- RF stops improving after 10,000 images.
- PCA reduces training time, inference time and memory, and improves k-NN.
- Macro-F1 is much lower than accuracy because of the weak minority classes.

Task 10 – Theory vs Experiment

k	Explained variance	k-NN accuracy	RF accuracy
5	0.5513	0.4736	0.5117
20	0.7093	**0.6358**	**0.6197**
50	0.7994	0.6200	0.6164
384	0.9500	0.4934	0.5106



1. **Explained variance vs accuracy:** only partly agrees. Accuracy peaks at k = 20, then drops even though more variance is kept. Variance measures reconstruction, not class information; the extra components are mostly noise.
2. **Optimisation speed vs condition number:** agrees in direction (whitened features train faster and more stably). The gap was smaller than $\kappa = 113$ suggests, because most of the loss sits in the top components, which converge fast.
3. **Bias–variance vs capacity curves:** agree. Low-capacity models have high bias and a small gap; high-capacity models have high variance and a large gap. One difference: k = 1 has lower 0-1 error than k = 100 despite higher squared error, because 0-1 loss only checks the top prediction.

Task 11 – Is Deep Learning Justified?

Limitations of the best baseline (RF raw, 0.6630):

- No gain from more data (0.6640 at 10,000 vs 0.6630 at 18,160).
- Early blight recall 0.1067, Mosaic 0.0000.
- Not robust to small changes:

Change	Accuracy
None	0.6608
Shift 4 pixels	0.5672
Horizontal flip	0.5668
Brightness +0.10	0.3069

(0.6608 differs slightly from 0.6630 because this run used unstandardised pixels.)

Curse of dimensionality, smoothness and manifold:

- **Curse of dimensionality:** the nearest/mean distance ratio is 0.4988 in raw space vs 0.3818 with PCA-50, so distances are less meaningful in raw space.
- **Local constancy:** the nearest neighbour has the same label only 47.91% of the time. A 4-pixel shift moves an image further (7.1443) than the distance to the nearest different training image (6.2486).
- **Manifold:** the intrinsic dimension is only 22.4760, but linear PCA needs 384 components for 95% variance, so the data lies on a curved low-dimensional surface.

Evidence for a learned hierarchical representation:

- PCA's main components capture lighting and position, not disease.
- Disease cues are local textures that can appear anywhere; convolution and pooling handle exactly this.
- The baseline fails under shifts and brightness changes.

Trade-offs: a CNN needs more data (or transfer learning), GPU training, and tools like Grad-CAM for interpretability.

Conclusion: deep learning (a CNN) is justified because classical models plateau, are not robust to shifts or lighting, and cannot learn local texture features. It is not needed for the numerical or imbalance issues, which can be fixed directly.

Task 12 – Day-4 Design Challenge (38 classes, 20% noisy labels, 50 ms / 5 MB)

Setup: all 38 PlantVillage classes (up to 400 images per class → 14,731 images), 20% of training labels flipped randomly, clean test labels.

A. Impact Analysis

	10 classes	**38 classes**
Entropy H(p)	3.0300 bits	4.8249 bits
Max entropy / initial loss $\log_2 K$	3.3219 bits	5.2479 bits
Majority-class accuracy	0.2950	0.1014
Best model accuracy	0.6608	0.4896

Noise floor: $H = -(1-\eta)\log_2(1-\eta) - \eta \log_2(\eta/(K-1)) = 0.2575 + 0.2 \times 7.5314 = 1.7638$ bits for $K = 38$.

- **Effect of noise on variance:** flexible models memorise wrong labels (1-NN and RF memorised about 99.9% of them), which adds variance.
- **Effect of noise on estimated error:** a noisy validation set overstates the error (true 0.5104 → observed 0.6056), so a clean validation set is needed.
- **Assumptions that no longer hold:** labels are correct; memory and time don't matter; accuracy is the only goal.

B. Examination of the Original Conclusion

Model	Accuracy	Latency (server)	Est. phone (×10)	Size	Fits?
RF-100 raw (best)	0.4896	5.574 ms	~56 ms	7.957 MB	No
k-NN PCA-50	0.4208	1.654 ms	~17 ms	2.789 MB	Yes
LogReg PCA-50	0.3656	0.877 ms	~9 ms	0.820 MB	Yes

- The best model does not fit the limits.
- Most sensitive to noise: k-NN with k = 1 (accuracy −0.0810).
- Least affected: Random Forest (only −0.0109), because voting over 100 trees averages out the noise.

C. Adaptation Options

Option	Effect on capacity	Effect on accuracy	Cost
PCA k = 50, regularised LogReg	Lower	0.3520	0.789 MB, <1 ms
Label smoothing (ε = 0.1–0.4)	Same	No gain (0.3507 → 0.3389)	Free
Compressed RF (50 trees, depth 16)	Lower	0.4068	3.028 MB, 2.6 ms
float16 quantisation	Same	0.3507 → 0.3511	Halves memory

Label smoothing did not help because the linear model was too simple to memorise the noise (it fitted only 2.7% of the wrong labels). It would help a high-capacity model such as a CNN.

D. Conclusion: MODIFY

- **Keep:** a CNN is still justified. With 38 classes the uncertainty is higher (4.8249 bits), and the classical models' limitations remain.
- **Change:**
  - Use a small, quantised CNN (roughly 1–3 MB).
  - Train it with label smoothing or a noise-robust loss, data augmentation, and class weights.
  - Validate on a clean set.
- **Interim fallback:** the compressed RF (3.03 MB, 0.4068 accuracy) fits the budget..

Appendix A – Code

```
import numpy as np, pickle, gzip, time, os, warnings, json
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
from PIL import Image
from common import *
warnings.filterwarnings("ignore")

ALL = sorted(os.listdir(DATA_ROOT))
print("Number of PlantVillage classes found:", len(ALL))
X38, y38, _ = load_folder_images(ALL, max_per_class=400,
cache_name="pv38_400")
print("38-class working set:", X38.shape, " per-class counts min/max:",
np.bincount(y38).min(), np.bincount(y38).max())
full_counts = np.array([len(os.listdir(os.path.join(DATA_ROOT, c))) for c in
ALL])
p38 = full_counts / full_counts.sum()
H = lambda p: -np.sum(p[p > 0] * np.log2(p[p > 0]))
p10 = np.bincount(load_tomato()[1]) / 18160
print("=" * 70, "\nPART A - label space\n" + "=" * 70)
print(f"Full PlantVillage: N={full_counts.sum()}  H(p38)={H(p38):.4f}
bits  log2(38)={np.log2(38):.4f}  majority acc={p38.max():.4f}")
print(f"Tomato 10-class:
H(p10)={H(p10):.4f}  log2(10)={np.log2(10):.4f}  majority
acc={p10.max():.4f}")
print(f"Initial cross-entropy (uniform output) 10 classes: {np.log2(10):.4f}
bits ({np.log(10):.4f} nats); 38 classes: {np.log2(38):.4f} bits
({np.log(38):.4f} nats)")
eta = 0.2
for K in (10, 38):
    # symmetric noise: with prob eta the label is replaced by one of the other
K-1 classes
    Hn = -(1 - eta) * np.log2(1 - eta) - eta * np.log2(eta / (K - 1))
    print(f"K={K}: entropy of noisy label given true class (irreducible CE
floor) = {Hn:.4f} bits; "
          f"best achievable accuracy on noisy labels = {1-eta:.2f}; random-
guess accuracy = {1/K:.4f}")

def noisy(yy, K, eta, seed):
    r = np.random.default_rng(seed); yn = yy.copy()
    flip = r.random(len(yy)) < eta
    yn[flip] = (yy[flip] + r.integers(1, K, flip.sum())) % K
    return yn, flip

def evaluate(Xa, ya, K, tag):
    Xtr, Xva, Xte, ytr, yva, yte = split(Xa, ya)
    sc = StandardScaler().fit(Xtr); pca = PCA(50,
random_state=SEED).fit(sc.transform(Xtr))
    Ztr, Zte = pca.transform(sc.transform(Xtr)),
pca.transform(sc.transform(Xte))
    ytr_n, flip = noisy(ytr, K, eta, SEED)
```

```
    print(f"[{tag}] train={len(ytr)} test={len(yte)}  actual noise
rate={flip.mean():.4f}")
    cfg = [("LogReg PCA-50", lambda: LogisticRegression(max_iter=500), "pca"),
           ("kNN k=5 PCA-50", lambda: KNeighborsClassifier(5, n_jobs=2),
"pca"),
           ("kNN k=25 PCA-50", lambda: KNeighborsClassifier(25, n_jobs=2),
"pca"),
           ("kNN k=1 PCA-50", lambda: KNeighborsClassifier(1, n_jobs=2),
"pca"),
           ("RF PCA-50", lambda: RandomForestClassifier(100, n_jobs=2,
random_state=SEED), "pca"),
           ("RF raw", lambda: RandomForestClassifier(100, n_jobs=2,
random_state=SEED), "raw")]
    out = {}
    for name, mk, f in cfg:
        A, B = (Ztr, Zte) if f == "pca" else (Xtr, Xte)
        mc = mk().fit(A, ytr); mn = mk().fit(A, ytr_n)
        ac, an = mc.score(B, yte), mn.score(B, yte)
        tr_fit = np.mean(mn.predict(A[flip]) == ytr_n[flip])      # how much
of the noise was memorised
        out[name] = (ac, an)
        print(f"  {name:16s} clean-label acc={ac:.4f}  20%-noisy
acc={an:.4f}  drop={ac-an:.4f}  "
              f"memorised wrong labels={tr_fit:.4f}")
    return out

print("=" * 70, "\nPART B - accuracy under 10 vs 38 classes, clean vs 20%
noisy training labels (clean test labels)\n" + "=" * 70)
Xt, yt, _ = load_tomato()
r10 = evaluate(Xt, yt, 10, "tomato 10-class")
r38 = evaluate(X38, y38, 38, "PlantVillage 38-class")

print("=" * 70, "\nPART B - latency (single image, batch=1, n_jobs=1) and
memory on this CPU\n" + "=" * 70)
Xtr, Xva, Xte, ytr, yva, yte = split(X38, y38)
sc = StandardScaler().fit(Xtr); pca = PCA(50,
random_state=SEED).fit(sc.transform(Xtr))
Ztr = pca.transform(sc.transform(Xtr))
Wp = (pca.components_ / sc.scale_).astype(np.float32)               # fold
scaler into PCA: z = Wp (x - m)
cen = (sc.mean_ + 0).astype(np.float32); pmean = pca.mean_.astype(np.float32)
def project(x):  # fused standardise + PCA, float32
    return ((x - cen) / sc.scale_.astype(np.float32) - pmean) @
pca.components_.T.astype(np.float32)
assert np.allclose(project(Xte[:5]), pca.transform(sc.transform(Xte[:5])),
atol=1e-3)
jpg = os.path.join(DATA_ROOT, ALL[0],
sorted(os.listdir(os.path.join(DATA_ROOT, ALL[0])))[0])
```

```
def preprocess(path):
    im = Image.open(path).convert("L").resize((64, 64), Image.BILINEAR)
    return (np.asarray(im, np.float32).reshape(1, -1) / 255)
def timeit(f, reps=200):
    f(); ts = []
    for _ in range(reps):
        t = time.perf_counter(); f(); ts.append(time.perf_counter() - t)
    return 1000 * np.median(ts)
t_pre = timeit(lambda: preprocess(jpg)); t_proj = timeit(lambda:
project(Xte[:1]))
print(f"Preprocess (decode JPEG + resize + scale): {t_pre:.3f} ms   PCA
projection: {t_proj:.3f} ms")
x1 = Xte[:1]; z1 = project(x1)
models = {
    "LogReg PCA-50": (LogisticRegression(max_iter=500).fit(Ztr, ytr), "pca"),
    "kNN k=5 PCA-50": (KNeighborsClassifier(5, n_jobs=1).fit(Ztr, ytr),
"pca"),
    "RF-100 PCA-50": (RandomForestClassifier(100, n_jobs=1,
random_state=SEED).fit(Ztr, ytr), "pca"),
    "RF-100 raw": (RandomForestClassifier(100, n_jobs=1,
random_state=SEED).fit(Xtr, ytr), "raw"),
    "RF-30 depth12 PCA-50": (RandomForestClassifier(30, max_depth=12,
n_jobs=1, random_state=SEED).fit(Ztr, ytr), "pca"),
}
Zte = project(Xte)
pca_bytes = Wp.nbytes + cen.nbytes + pmean.nbytes
print(f"PCA projection matrix 50x4096 float32 + means = {pca_bytes/2**20:.3f}
MB")
lat = {}
for name, (m, f) in models.items():
    inp = z1 if f == "pca" else x1
    t = timeit(lambda: m.predict(inp), reps=100)
    acc = m.score(Zte if f == "pca" else Xte, yte)
    sz = len(pickle.dumps(m)); gz = len(gzip.compress(pickle.dumps(m)))
    extra = pca_bytes if f == "pca" else 0
    if isinstance(m, LogisticRegression): lean = (m.coef_.size +
m.intercept_.size) * 4
    elif isinstance(m, KNeighborsClassifier): lean = Ztr.size * 4 + ytr.size
    else: lean = sum(e.tree_.node_count * 12 + e.tree_.n_leaves * 2 for e in
m.estimators_)  # int16 feat, f32 thr, 2x int16 child + leaf class id
    total_ms = t_pre + (t_proj if f == "pca" else 0) + t
    lat[name] = (acc, total_ms, (lean + extra) / 2**20)
    print(f"{name:22s} acc={acc:.4f}  predict={t:.3f} ms  end-to-
end={total_ms:.3f} ms  "
          f"pickle={sz/2**20:.3f} MB  gzip={gz/2**20:.3f} MB  lean
model+PCA={(lean+extra)/2**20:.3f} MB")
print("NOTE: timings are on this server CPU; a low-cost phone CPU is assumed
~5-10x slower.")
```

```
print("=" * 70, "\nPART C - adaptation options on 38 classes with 20% noisy
labels\n" + "=" * 70)
ytr_n, flip = noisy(ytr, 38, eta, SEED)
print("(C1) PCA dimension x regularisation for softmax regression (noisy
train, clean test):")
for k in (20, 50, 100):
    pk = PCA(k, random_state=SEED).fit(sc.transform(Xtr))
    A, B = pk.transform(sc.transform(Xtr)), pk.transform(sc.transform(Xte))
    for C in (0.01, 0.1, 1.0):
        m = LogisticRegression(C=C, max_iter=500).fit(A, ytr_n)
        print(f"   k={k:3d} C={C:<5} train(noisy) acc={m.score(A, ytr_n):.4f}
test acc={m.score(B, yte):.4f} params={(k+1)*38}  "
              f"memory={((k+1)*38*4 + k*4096*4)/2**20:.3f} MB")

print("(C2) Label smoothing in from-scratch softmax regression (whitened PCA-
50, noisy labels):")
Zs = Ztr / Ztr.std(0); Bs = Zte / Ztr.std(0)
def train_ls(A, yy, eps, lr=0.5, epochs=60, l2=1e-4):
    K = 38; Yt = np.eye(K)[yy] * (1 - eps) + eps / K
    W = np.zeros((A.shape[1], K)); b = np.zeros(K); rng =
np.random.default_rng(SEED)
    for ep in range(epochs):
        order = rng.permutation(len(A))
        for s in range(0, len(A), 128):
            i = order[s:s + 128]
            S = A[i] @ W + b; S -= S.max(1, keepdims=True); P = np.exp(S); P
/= P.sum(1, keepdims=True)
            G = P - Yt[i]; W -= lr * (A[i].T @ G / len(i) + l2 * W); b -= lr *
G.mean(0)
    return W, b
for eps in (0.0, 0.1, 0.2, 0.4):
    W, b = train_ls(Zs, ytr_n, eps)
    S = Bs @ W + b; S -= S.max(1, keepdims=True); P = np.exp(S); P /= P.sum(1,
keepdims=True)
    acc = np.mean(P.argmax(1) == yte); conf = P.max(1).mean()
    ce = np.mean(-np.log2(P[np.arange(len(yte)), yte] + 1e-12))
    print(f"   eps={eps:.1f}: test acc={acc:.4f}  mean
confidence={conf:.4f}  test CE={ce:.4f} bits  "
          f"fit to flipped labels={np.mean((Zs[flip] @ W + b).argmax(1) ==
ytr_n[flip]):.4f}")
print("(C3) Compressing the random forest (PCA-50 input, noisy labels):")
for n_est, depth in [(100, None), (50, 16), (30, 12), (20, 10), (10, 8)]:
    m = RandomForestClassifier(n_est, max_depth=depth, n_jobs=1,
random_state=SEED).fit(Ztr, ytr_n)
    lean = sum(e.tree_.node_count * 12 + e.tree_.n_leaves * 2 for e in
m.estimators_)
    t = timeit(lambda: m.predict(z1), reps=50)
```

```
    print(f"   trees={n_est:3d} depth={str(depth):4s} test acc={m.score(Zte,
yte):.4f}  nodes={sum(e.tree_.node_count for e in m.estimators_):7d} "
          f"lean size={lean/2**20:.3f} MB (+PCA
{pca_bytes/2**20:.3f})  predict={t:.3f} ms")
print("(C4) float16 quantisation of softmax regression + PCA:")
m = LogisticRegression(C=0.1, max_iter=500).fit(Ztr, ytr_n)
W16 = m.coef_.astype(np.float16); P16 = pca.components_.astype(np.float16)
z16 = (((Xte - cen) / sc.scale_) - pmean).astype(np.float16) @ P16.T
acc16 = np.mean((z16.astype(np.float32) @ W16.T.astype(np.float32) +
m.intercept_).argmax(1) == yte)
print(f"   float32 acc={m.score(Zte, yte):.4f}  float16 acc={acc16:.4f}  size
float32={(W16.size+P16.size)*4/2**20:.3f}
MB  float16={(W16.size+P16.size)*2/2**20:.3f} MB")
```